

Red Hat Data Grid 8.6

Data Grid Spring Boot Starter

Use Data Grid with your Spring Boot project

Last Updated: 2026-06-03

Red Hat Data Grid 8.6 Data Grid Spring Boot Starter

Use Data Grid with your Spring Boot project

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Quickly get your Spring Boot project up and running with a set of managed transitive dependencies that include everything your Spring Boot project needs to seamlessly interact with Data Grid. Note that while the Data Grid Spring Boot starter gives you a convenient way to get started with Spring Boot it is optional. To use Data Grid with Spring Boot you can simply add the dependencies you want.

Table of Contents

RED HAT DATA GRID 3

DATA GRID DOCUMENTATION 4

DATA GRID DOWNLOADS 5

MAKING OPEN SOURCE MORE INCLUSIVE 6

CHAPTER 1. USING EMBEDDED CACHES 7
1.1. ADDING THE EMBEDDEDCACHEMANAGER BEAN 7
1.2. USING THE REACTIVE MODE WITH REACTOR 7
1.3. CACHE MANAGER CONFIGURATION BEANS 7
1.4. ENABLING SPRING CACHE SUPPORT 9

CHAPTER 2. USING REMOTE CACHES 10
2.1. SETTING UP THE REMOTECACHEMANAGER 10
2.2. USING THE REACTIVE MODE WITH REACTOR 10
2.2.1. Properties Files 10
hotrod-client.properties 10
application.properties 10
2.3. CONFIGURING MARSHALLING 11
2.4. CONFIGURING AND CREATING CACHES ON STARTUP FROM THE CLIENT 11
2.5. CACHE MANAGER CONFIGURATION BEANS 12
2.6. ENABLING SPRING CACHE SUPPORT 13
2.7. EXPOSING DATA GRID STATISTICS 14

CHAPTER 3. USING SPRING SESSION 16
3.1. ENABLING SPRING SESSION SUPPORT 16

CHAPTER 4. APPLICATION PROPERTIES 17

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. USING EMBEDDED CACHES

Embed Data Grid caches directly in your project for in-memory data storage.

1.1. ADDING THE EMBEDDEDCACHEMANAGER BEAN

Configure your application to use embedded caches.

Procedure

1. Add **infinispan-spring-boot3-starter-embedded** or **infinispan-spring-boot4-starter-embedded** to your project's classpath to enable Embedded mode.
2. Use the Spring **@Autowired** annotation to include an **EmbeddedCacheManager** bean in your Java configuration classes, as in the following example:

```
private final EmbeddedCacheManager cacheManager;

@Autowired
public YourClassName(EmbeddedCacheManager cacheManager) {
   this.cacheManager = cacheManager;
}
```

You are now ready to use Data Grid caches directly within your application, as in the following example:

```
cacheManager.getCache("testCache").put("testKey", "testValue");
System.out.println("Received value from cache: " +
cacheManager.getCache("testCache").get("testKey"));
```

1.2. USING THE REACTIVE MODE WITH REACTOR

Starting with Spring 6.1, reactive mode is supported to make use of caching within reactive applications. If you use **spring-boot-starter-webflux**, your application may block.

To enable the Data Grid reactive driver, specify the following property in **application.properties**:

```
infinispan.embedded.reactive=true
```

1.3. CACHE MANAGER CONFIGURATION BEANS

You can customize the Cache Manager with the following configuration beans:

- **InfinispanGlobalConfigurer**
- **InfinispanCacheConfigurer**
- **Configuration**
- **InfinispanConfigurationCustomizer** deprecated in Spring Boot 3, removed in Spring Boot 4
- **InfinispanGlobalConfigurationCustomizer**

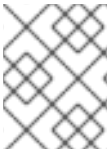


IMPORTANT

In Spring Boot 4, we removed the default cache **default** that could be customized with the **InfinispanConfigurationCustomizer**. Use **InfinispanCacheConfigurer** instead and define **default**.

```
@Bean
public InfinispanCacheConfigurer cacheConfigurer() {
 return manager -> {
  final Configuration ispnConfig = new ConfigurationBuilder()
                    .cacheMode(CacheMode.LOCAL)
                    .build();

  manager.defineConfiguration("default", ispnConfig);
 };
}
```



NOTE

You can create one **InfinispanGlobalConfigurer** bean only. However you can create multiple configurations with the other beans.

InfinispanCacheConfigurer Bean

```
@Bean
public InfinispanCacheConfigurer cacheConfigurer() {
 return manager -> {
  final Configuration ispnConfig = new ConfigurationBuilder()
                    .clustering()
                    .cacheMode(CacheMode.LOCAL)
                    .build();

  manager.defineConfiguration("local-sync-config", ispnConfig);
 };
}
```

Configuration Bean

Link the bean name to the cache that it configures, as follows:

```
@Bean(name = "small-cache")
public org.infinispan.configuration.cache.Configuration smallCache() {
    return new ConfigurationBuilder()
        .read(baseCache)
        .memory().size(1000L)
        .memory().evictionType(EvictionType.COUNT)
        .build();
}

@Bean(name = "large-cache")
public org.infinispan.configuration.cache.Configuration largeCache() {
    return new ConfigurationBuilder()
        .read(baseCache)
```

```
        .memory().size(2000L)
        .build();
}
```

Customizer Beans

```
@Bean
public InfinispanGlobalConfigurationCustomizer globalCustomizer() {
  return builder -> builder.transport().clusterName(CLUSTER_NAME);
}

@Bean
public InfinispanConfigurationCustomizer configurationCustomizer() {
  return builder -> builder.memory().evictionType(EvictionType.COUNT);
}
```

1.4. ENABLING SPRING CACHE SUPPORT

With both embedded and remote caches, Data Grid provides an implementation of Spring Cache that you can enable.

Procedure

- Add the **@EnableCaching** annotation to your application.

If the Data Grid starter detects the:

- **EmbeddedCacheManager** bean, it instantiates a new **SpringEmbeddedCacheManager**.
- **RemoteCacheManager** bean, it instantiates a new **SpringRemoteCacheManager**.

Reference

Spring Cache Reference

CHAPTER 2. USING REMOTE CACHES

Store and retrieve data from remote Data Grid clusters using Hot Rod, a custom TCP binary wire protocol.

2.1. SETTING UP THE REMOTECACHEMANAGER

Configure your application to use remote caches on Data Grid clusters.

1. Provide the addresses where Data Grid Server listens for client connections so the starter can create the **RemoteCacheManager** bean.

2. Use the Spring **@Autowired** annotation to include your own custom Cache Manager class in your application:

```
private final RemoteCacheManager cacheManager;

@Autowired
public YourClassName(RemoteCacheManager cacheManager) {
   this.cacheManager = cacheManager;
}
```

2.2. USING THE REACTIVE MODE WITH REACTOR

Starting with Spring 6.1, reactive mode is supported to make use of caching within reactive applications. If you use **spring-boot-starter-webflux**, your application may block.

To enable the Data Grid reactive driver, specify the following property in **application.properties**:

```
infinispan.remote.reactive=true
```

2.2.1. Properties Files

You can specify properties in either **hotrod-client.properties** or **application.properties**.

Properties can be in both properties files but the starter applies the configuration in **hotrod-client.properties** first, which means that file takes priority over **application.properties**.

hotrod-client.properties
Properties in this file take the format of **infinispan.client.hotrod.***, for example:

```
# List Data Grid servers by IP address or hostname at port localhost:11222.
infinispan.client.hotrod.server_list=127.0.0.1:11222
```

- Hot Rod client configuration API

application.properties
Properties in this file take the format of **infinispan.remote.***, for example:

```
# List Data Grid servers by IP address or hostname at port localhost:11222.
infinispan.remote.server-list=127.0.0.1:11222
```

Additional resources

- Application Properties

2.3. CONFIGURING MARSHALLING

Configure Data Grid to marshall Java objects into binary format so they can be transferred over the wire or stored to disk.

By default Data Grid uses a Java Serialization marshaller, which requires you to add your classes to an allow list. As an alternative you can use ProtoStream, which requires you to annotate your classes and generate a **SerializationContextInitializer** for custom Java objects.

Procedure

1. Open **hotrod-client.properties** or **application.properties** for editing.
2. Do one of the following:
   - Use ProtoStream as the marshaller.

     ```
     infinispan.client.hotrod.marshaller=org.infinispan.commons.marshall.ProtoStreamMarshalle
     ```

     ```
     infinispan.remote.marshaller=org.infinispan.commons.marshall.ProtoStreamMarshaller
     ```

   - Add your classes to the serialization allow list if you use Java Serialization. You can specify a comma-separated list of fully qualified class names or a regular expression to match classes.

     ```
     infinispan.client.hotrod.java_serial_allowlist=your_marshalled_beans_package.*
     ```

     ```
     infinispan.remote.java-serial-allowlist=your_marshalled_beans_package.*
     ```

3. Save and close your properties file.

Additional resources

- Cache Encoding and Marshalling

2.4. CONFIGURING AND CREATING CACHES ON STARTUP FROM THE CLIENT

Configure Data Grid to create caches on first access from the client.

Procedure

1. Open **hotrod-client.properties** or **application.properties** or **application.yaml** for editing.
2. Do one of the following:

```
infinispan.client.hotrod.cache.mycache.template_name=mytemplate1
infinispan.client.hotrod.cache.mycache.force_return_values=true
infinispan.client.hotrod.cache.mycache.marshaller=org.infinispan.commons.marshall.JavaSerial
zationMarshaller
```

```
infinispan.client.hotrod.cache.[com.myproject.mycache].template_name=mytemplate1
infinispan.client.hotrod.cache.
[com.myproject.mycache].marshaller=org.infinispan.commons.marshall.UTF8StringMarshaller

infinispan.client.hotrod.cache.[com.myproject.mycache].near_cache.mode=INVALIDATED
infinispan.client.hotrod.cache.[com.myproject.*].template_name=mytemplate2
infinispan.client.hotrod.cache.[com.myproject.*].transaction.transaction_mode=NON_XA
```

```
infinispan.remote.cache.example.configuration=<distributed-cache/>
infinispan.remote.cache.example.force-return-values=true
infinispan.remote.cache.mycache.template-name=myTemplate
infinispan.remote.cache.mycache.marshaller=org.infinispan.commons.marshall.JavaSerializatio
Marshaller
infinispan.remote.cache.[com.myproject.mycache].template-name=myCustomTemplate
infinispan.remote.cache.
[com.myproject.mycache].marshaller=org.infinispan.commons.marshall.UTF8StringMarshaller

infinispan.remote.cache.[com.myproject.mycache].near-cache.mode=INVALIDATED
infinispan.remote.cache.[com.myproject.*].template-name=myCustomTemplate
infinispan.remote.cache.[com.myproject.*].transaction.transaction-mode=NON_XA
```

```yaml
# Infinispan Configuration
infinispan:
  remote:
    # Basic Connection Settings
    server-list: "180.567.112.333:6668"

    # Cache Configurations
    cache:
      example:
        configuration: "<distributed-cache/>"
        force-return-values: true
      mycache:
        template-name: "myTemplate"
        marshaller: "org.infinispan.commons.marshall.JavaSerializationMarshaller"
      "[com.mycompany.mycache]":
        template-name: "mytemplate2"
        marshaller: "org.infinispan.commons.marshall.UTF8StringMarshaller"
        near-cache:
          mode: "INVALIDATED"
      "[com.mycompany.*]":
        template-name: "mytemplate2"
        transaction:
          transaction-mode: "NON_XA"
```

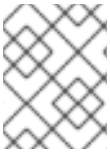
3. Save and close your properties or yaml file.

2.5. CACHE MANAGER CONFIGURATION BEANS

Customize the Cache Manager with the following configuration beans:

- **InfinispanRemoteConfigurer**
- **Configuration**

- **InfinispanRemoteCacheCustomizer**



NOTE

You can create one **InfinispanRemoteConfigurer** bean only. However you can create multiple configurations with the other beans.

InfinispanRemoteConfigurer Bean

```
@Bean
public InfinispanRemoteConfigurer infinispanRemoteConfigurer() {
   return () -> new ConfigurationBuilder()
      .addServer()
      .host("127.0.0.1")
      .port(12345)
      .build();
}
```

Configuration Bean

```
@Bean
public org.infinispan.client.hotrod.configuration.Configuration customConfiguration() {
   new ConfigurationBuilder()
      .addServer()
      .host("127.0.0.1")
      .port(12345)
      .build();
}
```

InfinispanRemoteCacheCustomizer Bean

```
@Bean
public InfinispanRemoteCacheCustomizer customizer() {
   return b -> b.tcpKeepAlive(false);
}
```

TIP

Use the **@Ordered** annotation to apply customizers in a specific order.

2.6. ENABLING SPRING CACHE SUPPORT

With both embedded and remote caches, Data Grid provides an implementation of Spring Cache that you can enable.

Procedure

- Add the **@EnableCaching** annotation to your application.

If the Data Grid starter detects the:

- **EmbeddedCacheManager** bean, it instantiates a new **SpringEmbeddedCacheManager**.

- **RemoteCacheManager** bean, it instantiates a new **SpringRemoteCacheManager**.

Reference

Spring Cache Reference

2.7. EXPOSING DATA GRID STATISTICS

Data Grid supports the Spring Boot Actuator to expose cache statistics as metrics.

Procedure

1. Add the following to your **pom.xml** file:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
  <version>${version.spring.boot}</version>
 </dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>${version.spring.boot}</version>
</dependency>
```

2. Activate statistics for the appropriate cache instances, either programmatically or declaratively.

   Programmatically

```
@Bean
public InfinispanCacheConfigurer cacheConfigurer() {
  return cacheManager -> {
    final org.infinispan.configuration.cache.Configuration config =
          new ConfigurationBuilder()
              .jmxStatistics().enable()
              .build();

    cacheManager.defineConfiguration("my-cache", config);
  };
}
```

   Declaratively

```
<local-cache statistics="true"/>
```

The Spring Boot Actuator registry binds cache instances when your application starts.

If you create caches dynamically, you should use the **CacheMetricsRegistrar** bean to bind caches to the Actuator registry, as follows:

```
@Autowire
CacheMetricsRegistrar cacheMetricsRegistrar;
```

```
@Autowire
CacheManager cacheManager;
...

cacheMetricsRegistrar.bindCacheToRegistry(cacheManager.getCache("my-cache"));
```

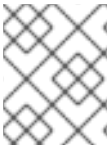
CHAPTER 3. USING SPRING SESSION

3.1. ENABLING SPRING SESSION SUPPORT

Data Grid Spring Session support is built on **SpringRemoteCacheManager** and **SpringEmbeddedCacheManager**. The Data Grid starter produces those beans by default.

Procedure

1. Add this starter to your project.
2. Add Spring Session to the classpath.
3. Add the following annotations to your configuration:
   - **@EnableCaching**
   - **@EnableInfinispanRemoteHttpSession**
   - **@EnableInfinispanEmbeddedHttpSession**



NOTE

Data Grid does not provide a default cache. To use Spring Session, you must first create a Data Grid cache.

CHAPTER 4. APPLICATION PROPERTIES

Configure your project with **application.properties** or **application.yaml**.

```
#
# Embedded Properties - Uncomment properties to use them.
#

# Enables embedded capabilities in your application.
# Values are true (default) or false.
#infinispan.embedded.enabled =

# Sets the Spring state machine ID.
#infinispan.embedded.machineId =

# Sets the name of the embedded cluster.
#infinispan.embedded.clusterName =

# Specifies a XML configuration file that takes priority over the global
# configuration bean or any configuration customizer.
#infinispan.embedded.configXml =

#
# Server Properties - Uncomment properties to use them.
#

# Specifies a custom filename for Hot Rod client properties.
#infinispan.remote.clientProperties =

# Enables remote server connections.
# Values are true (default) or false.
#infinispan.remote.enabled =

# Defines a comma-separated list of servers in this format:
# `host1[:port],host2[:port]`.
#infinispan.remote.server-list=

# Sets a timeout value, in milliseconds, for socket connections.
#infinispan.remote.socketTimeout =

# Sets a timeout value for initializing connections with servers.
#infinispan.remote.connectTimeout =

# Sets the maximum number of attempts to connect to servers.
#infinispan.remote.maxRetries =

# Specifies the marshaller to use.
#infinispan.remote.marshaller =

# Adds your classes to the serialization allow list.
#infinispan.remote.java-serial-allowlist=
```